

Binary Search Templates

Binary search appears in two forms: searching an **array index**, or searching an **answer value**.

Master Quick Reference

#	Name	Category	Template	i init	j init	Loop	Return
704	Binary Search	Array index	Closed	0	n-1	i <= j	-1
34	First & Last Position	Array index	Half-open	0	n	i < j	{lower, upper-1}
35	Search Insert Position	Array index	Half-open	0	n	i < j	i
33	Search in Rotated Array	Array index	Closed	0	n-1	i <= j	-1
162	Find Peak Element	Array index	Half-open*	0	n-1	i < j	i
875	Koko Eating Bananas	Answer space	MINIMIZE	1	max(piles)	i < j	i
1011	Ship Within D Days	Answer space	MINIMIZE	max(w)	sum(w)	i < j	i
410	Split Array Largest Sum	Answer space	MINIMIZE	max(nums)	sum(nums)	i < j	i
1283	Smallest Divisor	Answer space	MINIMIZE	1	max(nums)	i < j	i
1231	Divide Chocolate	Answer space	MAXIMIZE	min(s)	sum(s)/(k+1)	i < j	i

* #162 uses j = n-1 (not n) — loop reads arr[k+1], out of bounds if j = n.

All Four Templates

```
// 1. CLOSED [i, j] – exact match, return on hit
// MENTAL MODEL: target is a specific value sitting at some index; shrink a fully-closed range until you land on it.
// WHEN: "find this exact value in a sorted array"
int i = 0, j = n - 1;
while (i <= j) {
    int k = i + (j - i) / 2;
    if (arr[k] == target) {
        return k;
    } else if (arr[k] < target) {
        i = k + 1;
    } else {
        j = k - 1;
    }
}
return -1;

// 2. HALF-OPEN [i, j) – boundary search, answer falls out of i
// MENTAL MODEL: the predicate is false...false TRUE...true; find the FIRST position that satisfies the condition.
// WHEN: "first/last position", "insert position", "first index where condition holds"
int i = 0, j = n;          // j = n (or n-1 for #162)
while (i < j) {
    int k = i + (j - i) / 2;
    if (arr[k] < target) {
        i = k + 1; // lowerBound: <
    } else {
        j = k;      // upperBound: swap < for <=
    }
}
return i;

// 3. ANSWER SPACE MINIMIZE – smallest k where condition(k) is true
// MENTAL MODEL: condition goes false→TRUE as k grows (bigger k = more feasible); find the smallest k that flips it true.
// WHEN: "minimum X such that feasible(X)" – min speed, min capacity, smallest divisor
int i = minPossible, j = maxPossible;
while (i < j) {
    int k = i + (j - i) / 2;          // floor mid
    if (condition(k)) {
        j = k;
    } else {
        i = k + 1;
    }
}
return i;

// 4. ANSWER SPACE MAXIMIZE – largest k where condition(k) is true
// MENTAL MODEL: condition goes TRUE→false as k grows (bigger k = less feasible); find the largest k still true.
// WHEN: "maximum X such that feasible(X)" – max min-piece, max guarantee
int i = minPossible, j = maxPossible;
while (i < j) {
    int k = i + (j - i + 1) / 2;      // ceiling mid ← avoids infinite loop
    if (condition(k)) {
        i = k;
    } else {
        j = k - 1;
    }
}
return i;
```

How to Choose

Signal	Template
Sorted array, find exact value	Closed — return on <code>arr[k] == target</code>
First/last position, insert position	Half-open <code>lowerBound</code> / <code>upperBound</code>
Rotated array or slope search	Closed — pick sorted half each step
"minimum X such that feasible(X)"	Answer space MINIMIZE — floor mid
"maximum X such that feasible(X)"	Answer space MAXIMIZE — ceiling mid

Part 1 — Binary Search on Array Index

Detail Table

#	Name	Description	Interval	On <code>arr[k]==target</code>	Shrink left	Shrink right	Return
704	Binary Search	Find index of target; -1 if absent	Closed	<code>return k</code>	<code>i = k+1</code>	<code>j = k-1</code>	<code>-1</code>
34	First & Last Position	First and last index of target	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	<code>{lower, upper-1}</code>
35	Search Insert Position	Index where target is or would be inserted	Half-open	—	<code>i = k+1</code>	<code>j = k</code>	<code>i</code>
33	Search in Rotated Array	Find target after unknown-pivot rotation	Closed	<code>return k</code>	<code>i = k+1</code>	<code>j = k-1</code>	<code>-1</code>
162	Find Peak Element	Any local peak index	Half-open*	—	<code>i = k+1</code>	<code>j = k</code>	<code>i</code>

#704 Binary Search

Description: Given a sorted array and a target, return its index. Return -1 if not found.

Template: Closed `[i, j]` — return immediately on exact hit.

Intuition: The target is one specific value; halve a closed range until you sit on it (or the range empties).

```
class Solution {
    public int search(int[] arr, int target) {
        int i = 0, j = arr.length - 1;
        while (i <= j) {
            int k = i + (j - i) / 2;
            if (arr[k] == target) {
                return k;
            } else if (arr[k] < target) {
                i = k + 1;
            } else {
                j = k - 1;
            }
        }
        return -1;
    }
}
```

Time $O(\log n)$ | **Space** $O(1)$

#34 Find First and Last Position of Element in Sorted Array

Description: Return `[first, last]` index of target. Return `{-1, -1}` if absent.

Template: Half-open `[i, j)`. `lowerBound` vs `upperBound` differ by one char: `<` vs `<=`.

Intuition: Find the boundary where "`< target`" flips to "`≥ target`" (first position), and the next boundary just past target.

```

class Solution {
    public int[] searchRange(int[] arr, int target) {
        int i = lowerBound(arr, target);
        if (i == arr.length || arr[i] != target) return new int[]{-1, -1};
        return new int[]{i, upperBound(arr, target) - 1};
    }
    private int lowerBound(int[] arr, int target) {
        int i = 0, j = arr.length;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < target) {
                i = k + 1; // must go right
            } else {
                j = k; // k might be answer
            }
        }
        return i;
    }
    private int upperBound(int[] arr, int target) {
        int i = 0, j = arr.length;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] <= target) {
                i = k + 1; // still <= target, go right
            } else {
                j = k;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#35 Search Insert Position

Description: Return the index where target is or would be inserted to keep the array sorted.

Template: Half-open `[i, j)` — identical to `lowerBound` from #34.

Intuition: The insert position is the first index whose value is $\geq$ target — exactly `lowerBound`.

```

class Solution {
    public int searchInsert(int[] arr, int target) {
        int i = 0, j = arr.length;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < target) {
                i = k + 1;
            } else {
                j = k;
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

#33 Search in Rotated Sorted Array

Description: Array was sorted then rotated at an unknown pivot. Find target index, or -1.

Template: Closed `[i, j]`. One half is always sorted — identify which, then test if target lies in it.

Intuition: Even rotated, one half is always sorted; check if target lies in that half and discard the other.

```

class Solution {
    public int search(int[] arr, int target) {
        int i = 0, j = arr.length - 1;
        while (i <= j) {
            int k = i + (j - i) / 2;
            if (arr[k] == target) return k;
            if (arr[i] <= arr[k]) { // left half [i..k] is sorted
                if (arr[i] <= target && target < arr[k]) {
                    j = k - 1;
                } else {
                    i = k + 1;
                }
            } else { // right half [k..j] is sorted
                if (arr[k] < target && target <= arr[j]) {
                    i = k + 1;
                } else {
                    j = k - 1;
                }
            }
        }
        return -1;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

`arr[i] <= arr[k]` (not `<`) handles the edge case where `i == k`.

#162 Find Peak Element

Description: Return any index where `arr[k] > arr[k-1]` and `arr[k] > arr[k+1]`. Edges are $-\infty$.

Template: Half-open `[i, j)` with `j = n-1`. Binary search on slope: upward $\rightarrow$ peak is right, downward $\rightarrow$ peak is here or left.

Intuition: Always walk uphill — an upward slope guarantees a peak to the right, so the slope direction is the "condition".

```

class Solution {
    public int findPeakElement(int[] arr) {
        int i = 0, j = arr.length - 1;
        while (i < j) {
            int k = i + (j - i) / 2;
            if (arr[k] < arr[k + 1]) {
                i = k + 1; // slope up  $\rightarrow$  go right
            } else {
                j = k; // slope down  $\rightarrow$  peak at k or left
            }
        }
        return i;
    }
}

```

Time $O(\log n)$ | **Space** $O(1)$

Part 2 — Binary Search on Answer Space

Search on the **answer value** itself, not an array index. All use half-open `[i, j)` with `while (i < j)`.

Detail Table

#	Name	Description	Variant	i init	j init	Mid	Condition true	Condition false	Helper checks
875	Koko Eating Bananas	Min speed to finish all piles in h hours	MINIMIZE	1	max(piles)	floor	j = k	i = k+1	hours <= h
1011	Ship Within D Days	Min capacity to ship all packages in d days	MINIMIZE	max(w)	sum(w)	floor	j = k	i = k+1	days <= d
410	Split Array Largest Sum	Min largest sum splitting array into m parts	MINIMIZE	max(nums)	sum(nums)	floor	j = k	i = k+1	parts <= m
1283	Smallest Divisor	Smallest divisor so ceil-div-sum ≤ threshold	MINIMIZE	1	max(nums)	floor	j = k	i = k+1	divSum <= threshold
1231	Divide Chocolate	Max min-sweetness cutting into k+1 pieces	MAXIMIZE	min(s)	sum(s)/(k+1)	ceiling	i = k	j = k-1	pieces >= k+1

875 / 1011 / 410 / 1283 → identical structure, differ only in search bounds and condition helper.

1231 → only one that flips (MAXIMIZE): ceiling mid, i = k on success.

#875 Koko Eating Bananas

Description: Minimum eating speed so Koko finishes all piles in h hours (one pile per hour, at most speed bananas eaten).

Search space: [1, max(piles)]

Intuition: Feasibility is monotonic in speed (faster always works), so binary search the speed axis for the smallest that finishes in time.

```
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int i = 1, j = Arrays.stream(piles).max().getAsInt();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (canFinish(piles, k, h)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private boolean canFinish(int[] piles, int speed, int h) {
        int hours = 0;
        for (int p : piles) hours += (p + speed - 1) / speed;
        return hours <= h;
    }
}
```

Time O(n log(max(piles))) | **Space** O(1)

#1011 Capacity to Ship Packages Within D Days

Description: Minimum ship capacity to deliver all packages (in order) within d days.

Search space: [max(weights), sum(weights)] — must fit heaviest; worst case ships one per day.

Intuition: Bigger capacity is always feasible, so binary search capacity for the smallest that ships within the day budget.

```

class Solution {
    public int shipWithinDays(int[] weights, int days) {
        int i = Arrays.stream(weights).max().getAsInt();
        int j = Arrays.stream(weights).sum();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (canShip(weights, k, days)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private boolean canShip(int[] weights, int cap, int days) {
        int d = 1, load = 0;
        for (int w : weights) {
            if (load + w > cap) { d++; load = 0; }
            load += w;
        }
        return d <= days;
    }
}

```

Time $O(n \log(\text{sum}(\text{weights})))$ | **Space** $O(1)$

#410 Split Array Largest Sum

Description: Split array into m non-empty subarrays to minimize the largest subarray sum.

Search space: $[\max(\text{nums}), \text{sum}(\text{nums})]$ — identical bounds to #1011.

Intuition: A larger allowed max-sum needs fewer parts, so binary search that cap for the smallest splittable into $\leq m$ parts.

```

class Solution {
    public int splitArray(int[] nums, int m) {
        int i = Arrays.stream(nums).max().getAsInt();
        int j = Arrays.stream(nums).sum();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (canSplit(nums, k, m)) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private boolean canSplit(int[] nums, int cap, int m) {
        int parts = 1, sum = 0;
        for (int n : nums) {
            if (sum + n > cap) { parts++; sum = 0; }
            sum += n;
        }
        return parts <= m;
    }
}

```

Time $O(n \log(\text{sum}(\text{nums})))$ | **Space** $O(1)$

#1283 Find the Smallest Divisor Given a Threshold

Description: Smallest positive divisor such that $\text{sum of } \text{ceil}(\text{nums}[i] / \text{divisor}) \leq \text{threshold}$.

Search space: $[1, \max(\text{nums})]$

Intuition: A bigger divisor shrinks the sum, so the condition flips false→true; binary search for the smallest divisor that fits the threshold.

```
class Solution {
    public int smallestDivisor(int[] nums, int threshold) {
        int i = 1, j = Arrays.stream(nums).max().getAsInt();
        while (i < j) {
            int k = i + (j - i) / 2;
            if (divSum(nums, k) <= threshold) {
                j = k;
            } else {
                i = k + 1;
            }
        }
        return i;
    }
    private int divSum(int[] nums, int d) {
        int sum = 0;
        for (int n : nums) sum += (n + d - 1) / d;
        return sum;
    }
}
```

Time $O(n \log(\max(\text{nums})))$ | **Space** $O(1)$

#1231 Divide Chocolate ← MAXIMIZE (the one that differs)

Description: Cut chocolate into $k+1$ pieces; keep the minimum-sweetness piece. Maximize that minimum.

Search space: $[\min(s), \text{sum}(s)/(k+1)]$

Intuition: A larger target-minimum yields fewer pieces, so the condition flips true→false; binary search for the largest minimum still giving $\geq k+1$ pieces.

Ceiling mid is required. Concrete trace: with floor mid, $i=4, j=5 \rightarrow \text{mid}=4 \rightarrow$ on success $i=4 \rightarrow$ infinite loop; ceiling mid $i+(j-i+1)/2 \rightarrow \text{mid}=5 \rightarrow i=5 \rightarrow$ exits.

```
class Solution {
    public int maximizeSweetness(int[] s, int k) {
        int i = Arrays.stream(s).min().getAsInt();
        int j = Arrays.stream(s).sum() / (k + 1);
        while (i < j) {
            int m = i + (j - i + 1) / 2; // ceiling: guarantees progress when j == i+1
            if (canDivide(s, k + 1, m)) {
                i = m;
            } else {
                j = m - 1;
            }
        }
        return i;
    }
    private boolean canDivide(int[] s, int pieces, int minSweet) {
        int count = 0, sum = 0;
        for (int x : s) {
            sum += x;
            if (sum >= minSweet) { count++; sum = 0; }
        }
        return count >= pieces;
    }
}
```

Time $O(n \log(\text{sum}(s)))$ | **Space** $O(1)$

MINIMIZE vs MAXIMIZE Side by Side

	MINIMIZE (875,1011,410,1283)	MAXIMIZE (1231)
Mid:	$\text{floor } i + (j-i) / 2$	$\text{ceiling } i + (j-i+1) / 2$
If condition true:	$j = k$	$i = k$
If condition false:	$i = k + 1$	$j = k - 1$
Condition checks:	$\leq \text{budget}$ (feasible)	$\geq \text{count}$ (enough pieces)

Why ceiling for MAXIMIZE: when $i+1 == j$:

- floor gives $k = i \rightarrow$ if true, $i = i \rightarrow$ **infinite loop**
- ceiling gives $k = j \rightarrow$ if true, $i = j \rightarrow$ loop ends ✓